SafeRoute: A Crowdsourced Safety-Aware Navigation Web App

Project Proposal for UCS503
Submitted to: Dr. Paramveer Sidhu
Thapar Institute of Engineering and Technology

Dhriti Handa, COE* Disha Kundra, COE†

August 13, 2026

1 Higher-order goal

. . . secondary framing

Everyone who has ever walked home at night has done the same mental calculation: which street actually feels safe, not just which one is shortest. No navigation tool answers that question today. SafeRoute does. It's a web application – open a link, no install, no app store – that puts a live, crowdsourced safety score next to every route, so students, women commuting after work, and night-shift staff can choose the safer path with the same confidence they already choose the faster one. The bigger goal is simple: make safety a routing input, the same way Google Maps made traffic a routing input.

2 Time-to-value

. . . primary heuristic

- We will deliver a visibly working core loop early: the web app will show a live map with a seeded safety heatmap and a route that visibly differs between "fastest" and "safest" toggles.
- We will prioritize a usable, demoable increment over a fully general one – the first version uses manually seeded incident data before the crowdsourcing and ML layers are live, so the value of the safety toggle is demonstrable from the first prototype.
- Success in each iteration is defined by what can be shown end-to-end in a browser (navigation, reporting, heatmap), not by the completeness of any single subsystem.

3 Problem Statement

Conventional navigation platforms – Google Maps, Apple Maps – are excellent at optimizing for distance, travel time, and traffic. None of them give a user a dedicated, dynamically-computed safety layer to compare routes on. This gap matters most exactly when it's most dangerous to ignore: walking at night or through an unfamiliar area, where lighting and reported incidents can differ sharply between two streets that are otherwise equally fast.

*Roll No: 1024030176, Email: dhanda_be24@thapar.edu
†Roll No: 1024030180, Email: ddisha_be24@thapar.edu

- **No safety option, period:** the shortest path may run through a poorly-lit or isolated stretch, and neither the app nor the user has any way to know that in advance.
- **Safety tools exist, but don't navigate:** apps like Safetipin collect useful safety data, but they function as planning/audit dashboards for city governments – not as a consumer tool you'd actually open to get walking directions.
- **SOS tools exist, but don't route:** apps like bSafe or Noonlight handle emergencies well, but are purely reactive – they say nothing about which street to take in the first place, and require an install.
- **No time-of-day awareness:** a street that's fine at 6 pm can feel very different at 11 pm; mainstream routing tools don't capture this at all.

Why this matters right now: street lighting and safety perception vary block by block across Indian cities and campuses. This is something we and our classmates navigate every single day – it's a real, felt problem, not one we invented to fit a project brief, and it currently gets solved with word-of-mouth guesswork instead of any systematic tool.

4 Existing Solutions and the Gap We Fill

- **Google/Apple Maps:** best-in-class routing and real-time traffic, but zero safety-awareness – optimizes purely for time and distance.
- **Safetipin:** pioneered urban safety scoring in India and works with city partners, but it's a planning tool, not a navigation app – there's no "give me the safest route right now" experience, and coverage outside partner cities is thin.
- **bSafe / Noonlight:** strong SOS and live-location-sharing, but mobile-app-only, install-required, and entirely reactive – no routing component at all.

SafeRoute is the first to put these three ideas – live routing, crowdsourced safety data, and one-tap SOS – into a single, install-free web app, with a spatial model that fills in the gaps where crowd data doesn't exist yet.

5 Proposed Solution *. . . Software Proposition*

5.1 Overview

SafeRoute is a **responsive web application** that layers a dynamically-updated, crowdsourced safety score on top of standard map routing:

- **Turn-by-turn navigation:** standard walking directions via a self-hosted OSRM engine, rendered on a Mapbox GL JS map, right in the browser.
- **Fastest vs. Safest toggle:** one click re-ranks candidate routes by a safety-weighted cost function instead of distance alone.
- **One-click incident reporting:** geotagged reports (poor lighting, isolated area, past incident) using the browser's Geolocation API – minimal friction, no login wall to just look around.
- **Live safety heatmap:** a color-graded overlay built from real reports, historical density, and time of day.

- **Spatial interpolation for sparse areas:** the map is never blank – unreported streets get an estimated score based on nearby data.
- **SOS quick-share:** one click generates a live location link sent to a pre-set emergency contact.

5.2 Core workflow

1. User opens the app in any browser, picks a source and destination, and taps "fastest" or "safest."
2. The backend checks safety scores for candidate routes in PostGIS and re-ranks them with the safety-weighted cost function.
3. While reviewing the route, if the user knows of a hazard, they log it in two clicks – location captured automatically via the browser.
4. Nearby safety-score cells update, and the safety heatmap refreshes for the next person who opens that area.

6 Solution Approach *... Engineering focus*

This is an engineering course project, so our effort goes into geospatial systems integration and routing correctness – not into inventing new machine-learning research.

6.1 Safety-score interpolation *... non-research*

Estimating safety where reports are sparse is heuristic-and-model assisted, not a research contribution:

- Raw incident reports are aggregated into a PostGIS grid, weighted by recency and community upvotes.
- Inverse Distance Weighting (IDW) – a simple, interpretable spatial interpolation technique – fills in scores for grid cells with no direct reports, using nearby cells and time-of-day as inputs.
- Interpolated cells are always shown as lower-confidence than directly-reported ones, and the app never hides or blocks a route without the user's own choice.

6.2 System architecture *... web + geospatial*

A React web client talks to a Flask backend, which coordinates a geospatial database and a routing engine:

- **Web client:** React + Vite, with Mapbox GL JS for the map, incident reporting, the heatmap layer, and SOS – configured as an installable PWA.
- **Backend API:** Flask REST API handling auth, incident reports, safety-score queries, and safety-weighted routing requests.
- **Geospatial database:** PostgreSQL with PostGIS, storing grid-based safety scores and reports with spatial (GiST) indexing for fast radius/route queries.
- **Routing engine:** a self-hosted OSRM instance; the safety layer is applied either by modifying OSRM's cost profile, or – as a fallback if that proves too complex – by re-ranking OSRM's candidate routes using an externally computed safety score.

- **Realtime layer:** Firebase Auth for login and Firebase Cloud Messaging / Web Push for low-safety-zone alerts.
- **Batch interpolation job:** a periodic job recomputes the safety grid from new reports using the IDW model, keeping live queries fast.

6.3 Tech stack at a glance

Layer	Tools
Frontend	React, Vite, JavaScript, HTML/CSS, Mapbox GL JS
Backend	Python, Flask, REST APIs
Database	PostgreSQL + PostGIS
Routing	OSRM (self-hosted), Mapbox Directions & Geocoding APIs
Auth & Notifications	Firebase Authentication, Firebase Cloud Messaging / Web Push
AI/ML & Data	Python, NumPy, Pandas, scikit-learn / SciPy (IDW interpolation)
Dev Tools	Git, GitHub, VS Code, Postman
Deployment	Vercel/Netlify (frontend), Render/Railway (backend + OSRM), Supabase/Neon (PostGIS)

Every layer uses a mature, well-documented tool – we spend our time on routing correctness and geospatial performance, not on reinventing infrastructure that's already solved.

6.4 CI/CD and fast, safe delivery

- Every push runs backend tests (pytest) and frontend tests (Jest + React Testing Library) via GitHub Actions.
- Each milestone gets a repeatable frontend deploy (Vercel/Netlify) and staging backend deploy ahead of the demo.
- New features – reporting, heatmap, safest-route toggle, SOS – ship incrementally without breaking core navigation.

7 Evaluation Criterion

. . . measurable and attributable

We won't just claim SafeRoute works – we'll measure it.

7.1 Primary evaluation metric

Route divergence validity: does the "safest route" actually differ from the "fastest route" when safety scores along the corridor are clearly different?

- Target: on a seeded test map of our campus/demo area, safest and fastest routes visibly diverge in at least 80% of test cases containing a deliberately low-safety segment.
- Measurement: compare route geometry from the safety-weighted routing function against the plain, unweighted OSRM route for identical start/end points.

7.2 Secondary evaluation metrics

- **Report-to-heatmap latency:** time between a submitted report and the heatmap reflecting it (target: under 10 seconds).
- **Interpolation sanity check:** spot-check IDW-estimated scores against a small hand-labeled set of "ground truth" cells.
- **Query performance:** p95 latency for radius/route safety queries under simulated load.
- **Reliability:** uptime of the staging/production backend and OSRM instance during the demo window.

7.3 Pilot validation plan *. . . high-level*

- Get 10–20 peers on campus to seed the first round of reports and try the reporting flow.
- Run a live walkthrough: show fastest and safest routes diverging, submit a live report on stage, and watch the heatmap update in real time.

8 Scalability *. . . with foresight*

The system needs to hold up both in theory and in practice.

1. **Scales in theory:** the web client is fully decoupled from the backend and routing engine; PostGIS spatial indexing (GiST) replaces full-table scans; the backend API is stateless, so it can scale horizontally behind a load balancer.
2. **Deployable in practice:** React frontend on Vercel/Netlify (static hosting + CDN); Flask backend and OSRM on Render/Railway; managed PostGIS-enabled Postgres on Supabase or Neon; a GitHub Actions CI/CD pipeline for staging and production.

Beyond a single campus, city-scale coverage would need bigger OSRM map data and query infrastructure – a deliberate, honest "future scope" item, not something we're pretending is already solved.

9 Engine availability heuristic

Nothing here is built from scratch – every piece rests on a mature, well-supported tool:

- Web framework: React + Vite, for a fast, map-heavy single-page app.
- Routing: self-hosted OSRM with a modified cost profile (or route re-ranking as fallback), plus Mapbox for map tiles and geocoding.
- Geospatial storage: PostgreSQL + PostGIS with GiST indexing.
- Backend: Flask for the REST API.
- Auth and push: Firebase Authentication and Web Push / Firebase Cloud Messaging.
- Interpolation: scikit-learn / SciPy for a lightweight IDW implementation, not a custom research model.

10 Project Scope and Deliverables *. . . iteration-friendly*

10.1 Initial deliverable *. . . first iteration / prototype*

- Proposal finalization, wireframes, repository and architecture setup.
- Basic map and turn-by-turn navigation integration (Mapbox/OSRM) in the web app.
- Incident reporting feature and PostGIS schema for reports and the safety-score grid.
- **Prototype:** web app shows a map with a manually-seeded safety heatmap and a route that visibly differs between "fastest" and "safest" toggles, for our own campus/city.

10.2 Subsequent deliverables

- Dynamic safety-score computation from live reports and the safest-route toggle wired to the real cost function.
- Progress report; IDW-based interpolation model for sparse-data areas.
- SOS/emergency quick-share feature, push notifications for low-safety zones, and UX polish.
- Load testing of geospatial queries and bug fixes.
- Deployment of the backend, OSRM instance, and web app.
- Final presentation preparation and live demo rehearsal.

11 Risks and Mitigations

- **Cold start (no data for new areas):** manually seed safety scores for the campus/demo area up front; mark interpolated cells as lower-confidence.
- **Fake or malicious reports:** rate-limit reports per authenticated user, require a minimum upvote count before a report meaningfully moves the safety score.
- **OSRM cost-function complexity:** budget explicit research time in Week 3–4; fallback is re-ranking OSRM's candidate routes by an externally computed safety score, rather than editing OSRM's Lua profile internally.
- **Browser geolocation limits:** permission prompts and desktop IP-based accuracy are worse than mobile GPS – mitigated with manual address entry as a graceful fallback.
- **Slow queries at scale:** PostGIS GiST indexes from day one, load-tested in Week 13, well before deployment.
- **Two people, three domains (frontend, geospatial backend, ML):** strict scope discipline – PWA polish and push notifications are stretch goals, and the Week 7 prototype scope is protected above everything else.

12 Summary

This project proposes a deployable, web-based, safety-aware navigation application that layers crowdsourced and interpolated safety scores on top of standard shortest-path routing. It meets the course criteria by emphasizing time-to-value through a Week-7 demoable prototype, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria

(route divergence validity, report-to-heatmap latency, and geospatial query performance). It remains focused on systems-engineering depth – full-stack web development, geospatial databases, and applied spatial interpolation – rather than research-driven algorithmic novelty.